

CFD from Scratch: A Self-Paced Python Course from Finite Differences to Turbulence

04 June 2026

Summary

CFD from Scratch is an open-source, self-paced learning module that guides students through Computational Fluid Dynamics (CFD) from first principles using Python and Jupyter notebooks. Starting from the definition of a finite difference and ending with industrial turbulence models (k-epsilon and k-omega SST), the course is structured around a write-it-yourself pedagogy: every numerical method is derived, then immediately implemented by the learner in a paired exercise notebook.

The module spans four progressively deeper areas: (1) numerical foundations — finite differences, 1D advection, diffusion, and Burgers' equation; (2) core CFD methods — 2D scalar transport, the Finite Volume Method, incompressible Navier-Stokes, pressure-velocity coupling, and the SIMPLE algorithm; (3) advanced topics — RANS turbulence modeling, k-epsilon, k-omega SST, and mesh generation; and (4) project-scale simulations — channel flow, heat exchangers, airfoils, and turbulent jets. The lid-driven cavity solver is validated against the Ghia et al. benchmark (Ghia et al. 1982).

All content is hosted at github.com/harishragul/cfd under the MIT license and developed in the open as an ongoing learning record.

Statement of Need

Learners who want to understand what a CFD solver actually does — not just how to run one — face a gap in available resources. Introductory courses teach fluid mechanics theory; commercial training teaches software operation. Resources that build solvers from scratch, such as the widely-used *CFD Python* (Barba and Forsyth 2018), cover the foundational 12 steps through basic Navier-Stokes but stop before turbulence modeling, which is where nearly all real-world engineering CFD begins.

This module fills that gap. It takes a learner from their first finite difference

stencil all the way to implementing k-omega SST (Menter 1994) — the turbulence model used by default in OpenFOAM, Fluent, and Star-CCM+ — without ever using a black-box solver. Every discretization, every boundary condition, and every iterative pressure correction is written by the learner in NumPy.

The course is particularly suited to:

- Engineering graduates preparing for research or industry CFD work who want conceptual depth beyond software familiarity
- Students in numerical methods or fluid mechanics courses seeking a coding companion
- Self-directed learners who learn by building rather than reading

Learning Objectives

By completing this module, students will be able to:

1. Derive finite difference approximations and verify their order of accuracy using convergence plots
2. Implement and stability-analyze explicit and implicit time-integration schemes for advection and diffusion
3. Explain the CFL condition and von Neumann stability criterion and apply them to choose time steps
4. Implement the Finite Volume Method with exact flux conservation and verify mass conservation numerically
5. Write a pressure-velocity coupled Navier-Stokes solver using the projection method and SIMPLE algorithm
6. Validate a CFD solver against a published benchmark (Ghia et al., 1982)
7. Explain the closure problem in turbulent flow and implement mixing-length, k-epsilon, and k-omega SST models
8. Design a wall-resolved mesh using y^+ estimation and geometric clustering

Instructional Design

The module applies two complementary pedagogical strategies throughout.

Socratic sequencing. Each topic opens with a question the student is expected to answer from physical intuition before any equation is introduced. For example, the advection module begins: *“If information travels rightward, which neighboring cell should you look at to estimate the flux?”* This question-first structure is implemented in the reference notebooks as guided discovery rather than exposition.

Feynman paired exercises. After each reference notebook explains a concept, the student opens a paired exercise notebook in the **exercise/** folder and implements the method independently. The exercise notebooks provide function signatures and docstrings; all numerical content is written by the student. This

mirrors the Feynman technique: understanding is demonstrated by being able to reconstruct the result, not by being able to follow it.

The content ordering follows a physical-to-numerical progression: physical analogy $\rightarrow$ governing equation $\rightarrow$ discretization $\rightarrow$ implementation $\rightarrow$ validation. This sequence was found to be more effective for building robust mental models than leading with the mathematics (Ferziger et al. 2020).

The reference notebooks in `curriculum/` were generated by an AI tutor (Claude Code) acting as an interactive Socratic instructor, asking questions before explaining and correcting mistakes without giving away answers. The exercise notebooks in `exercise/` are the student’s own implementations, written during those sessions.

Content Description

The module is organized into four directories:

- `curriculum/module_01_foundations/` (8 notebooks): finite differences, 1D linear advection, 1D diffusion (explicit FTCS, implicit BTCS, Crank-Nicolson), and Burgers’ equation
- `curriculum/module_02_core_methods/` (8 notebooks): 2D scalar transport, Finite Volume Method, incompressible Navier-Stokes, pressure-velocity coupling, the SIMPLE algorithm (Patankar 1980), lid-driven cavity, flow over a cylinder, and boundary conditions
- `curriculum/module_03_advanced/` (8 notebooks): RANS equations, mixing-length model, k-epsilon model, k-omega SST, mesh generation (structured, unstructured, hybrid), higher-order schemes, multigrid methods, unsteady flows, compressible flow, and Physics-Informed Neural Networks
- `exercise/` (13 notebooks): student implementations of all major solvers, from 1D advection through the full lid-driven cavity SIMPLE solver and turbulent channel flow mesh design

The lid-driven cavity solver (41×41 grid, $Re = 100$) converges in approximately 610 time steps and produces u- and v-velocity profiles that agree well with Ghia et al. (Ghia et al. 1982), as shown in `results/ghia_validation.png`.

Evidence of Use

This module was used by the author as their primary self-study resource for CFD over a period of approximately three months (March–June 2026). Learning progress, quiz results, and common misconceptions are documented in `LEARNING_PROGRESS.md`, which serves as a detailed record of the learning trajectory. Module assessments (multiple-choice and implementation quizzes) were administered after each module: Module 1 — 8.5/10; Module 2 — 10/12

(83%); Module 3 (partial) — turbulence models 17/20 (85%). The module is actively shared publicly for adoption by other self-directed learners.

Acknowledgements

The reference notebooks and teaching protocol were developed with the assistance of the Claude Code AI agent (Anthropic), which acted as an interactive Socratic tutor throughout the course. All solver implementations in the `exercise/` folder were written independently by the author.

References

- Barba, Lorena A., and Gilbert F. Forsyth. 2018. “CFD Python: The 12 Steps to Navier-Stokes Equations.” *Journal of Open Source Education* 2 (16): 21. <https://doi.org/10.21105/jose.00021>.
- Ferziger, Joel H., Milovan Perić, and Robert L. Street. 2020. *Computational Methods for Fluid Dynamics*. 4th ed. Springer. <https://doi.org/10.1007/978-3-319-99693-6>.
- Ghia, U., K. N. Ghia, and C. T. Shin. 1982. “High-Re Solutions for Incompressible Flow Using the Navier-Stokes Equations and a Multigrid Method.” *Journal of Computational Physics* 48 (3): 387–411. [https://doi.org/10.1016/0021-9991\(82\)90058-4](https://doi.org/10.1016/0021-9991(82)90058-4).
- Menter, Florian R. 1994. “Two-Equation Eddy-Viscosity Turbulence Models for Engineering Applications.” *AIAA Journal* 32 (8): 1598–605. <https://doi.org/10.2514/3.12149>.
- Patankar, Suhas V. 1980. *Numerical Heat Transfer and Fluid Flow*. Hemisphere Publishing Corporation.